

ui.color_picker 全面详解

`ui.color_picker` 是 NiceGUI 框架中基于 Quasar 的 QMenu（菜单）和 QColor（颜色选择）组件封装的弹窗式颜色选择器，核心特点是通过触发元素（如按钮）打开独立的颜色选择菜单，支持颜色拾取回调、菜单状态控制、自定义外观等功能，适用于需要节省界面空间、提供沉浸式颜色选择体验的场景（如工具栏颜色设置、快捷样式调整等）。

一、核心初始化参数

初始化 `ui.color_picker` 时仅需关注两个核心参数，简洁高效地实现基础颜色选择功能：

参数名	说明
<code>on_pick</code>	颜色拾取完成时触发的回调函数，事件对象通过 <code>e.color</code> 获取选中的颜色值（支持标准颜色格式）
<code>value</code>	菜单初始状态是否打开，布尔值（默认 <code>False</code> ，设为 <code>True</code> 时组件渲染后自动展开菜单）

基础使用示例

```
from nicegui import ui

# 以按钮为触发元素，嵌套颜色选择器
with ui.button(icon='colorize', label='选择颜色') as trigger_btn:
    # 初始化颜色选择器
    ui.color_picker(
        on_pick=lambda e: (
            # 颜色拾取后，将按钮背景设置为选中颜色
            trigger_btn.classes(f'!bg-[{e.color}]'),
            ui.notify(f'选中颜色: {e.color}') # 弹出通知提示
        )
    )

ui.run()
```

二、组件核心属性

`ui.color_picker` 继承 NiceGUI 基础元素的核心属性，支持样式自定义、状态绑定、DOM 标识等能力，关键属性如下：

属性名	类型	说明	
<code>classes</code>	<code>Classes[Self]</code>	组件的 HTML 类名，用于通过 Tailwind/Quasar 样式自定义菜单外观	
<code>client</code>	<code>Client</code>	组件所属的客户端实例（多客户端场景下的隔离标识）	

属性名	类型	说明	
<code>html_id</code>	<code>str</code>	组件在 HTML DOM 中的唯一 ID (2.16.0 版本新增, 用于精准 DOM 操作)	
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除 (只读属性, 用于判断组件生命周期状态)	
<code>is_ignoring_events</code>	<code>bool</code>	组件是否忽略事件 (只读属性, 用于调试或事件控制场景)	
<code>parent_slot</code>	<code>`Slot`</code>	<code>None`</code>	组件的父插槽 (可设置, 用于复杂布局中的插槽嵌套)
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 原生属性 (用于扩展菜单本身的行为, 如菜单位置、动画等)	
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式 (如设置菜单宽度、边框等)	
<code>value</code>	<code>BindableProperty</code>	菜单的当前状态 (打开 / 关闭, <code>True</code> 为打开, <code>False</code> 为关闭, 支持绑定)	
<code>visible</code>	<code>BindableProperty</code>	组件是否可见 (支持动态绑定, <code>False</code> 时隐藏整个颜色选择器及触发逻辑)	

属性操作示例

```
# 初始化颜色选择器并绑定菜单状态
color_picker = ui.color_picker(value=False)

# 按钮控制菜单状态
ui.button('打开菜单', on_click=lambda: color_picker.set_value(True))
ui.button('关闭菜单', on_click=lambda: color_picker.set_value(False))
ui.button('隐藏选择器', on_click=lambda: color_picker.set_visibility(False))
```

三、核心方法

`ui.color_picker` 提供丰富的方法用于菜单控制、颜色操作、事件绑定等, 按功能分类如下:

1. 菜单状态控制方法

核心用于控制颜色选择菜单的打开、关闭、切换, 是组件交互的核心能力:

方法名	说明
<code>open()</code>	主动打开颜色选择菜单（无需点击触发元素）
<code>close()</code>	主动关闭颜色选择菜单（如拾取颜色后自动关闭）
<code>toggle()</code>	切换菜单状态（当前打开则关闭，当前关闭则打开）
<code>set_value(value: bool)</code>	设置菜单状态（ <code>True</code> 打开， <code>False</code> 关闭，与直接赋值 <code>color_picker.value = ...</code> 等价）

菜单控制示例

```
from nicegui import ui

# 独立按钮触发，而非嵌套在按钮内
ui.button('打开颜色选择器', on_click=lambda: picker.open())

# 初始化颜色选择器（无默认触发元素，通过方法控制菜单）
picker = ui.color_picker(
    on_pick=lambda e: (
        ui.notify(f'选中颜色: {e.color}'),
        picker.close() # 拾取后自动关闭菜单
    )
)

ui.run()
```

2. 颜色操作方法

用于直接设置颜色选择器的当前颜色，支持动态同步颜色状态：

方法名	说明
<code>set_color(color: str)</code>	设置颜色选择器的当前选中颜色（支持十六进制、RGB、颜色名等标准格式）

颜色操作示例

```
picker = ui.color_picker(on_pick=lambda e: ui.notify(f'选中颜色: {e.color}'))

# 按钮设置预设颜色
ui.button('设置为红色', on_click=lambda: picker.set_color('#ff0000'))
ui.button('设置为蓝色', on_click=lambda: picker.set_color('rgb(0, 0, 255)'))
ui.button('设置为绿色', on_click=lambda: picker.set_color('green'))
```

3. 绑定相关方法

支持将菜单状态（value）、组件可见性（visible）与目标对象属性进行单向 / 双向绑定，实现数据同步：

方法名	说明
<code>bind_value(target_object, ...)</code>	双向绑定菜单状态（打开 / 关闭）与目标对象属性
<code>bind_value_from(...)</code>	单向绑定（目标对象 → 组件）菜单状态
<code>bind_value_to(...)</code>	单向绑定（组件 → 目标对象）菜单状态
<code>bind_visibility(...)</code>	双向绑定组件可见性与目标对象属性
<code>bind_visibility_from(...)</code>	单向绑定（目标对象 → 组件）可见性
<code>bind_visibility_to(...)</code>	单向绑定（组件 → 目标对象）可见性

绑定示例

```
from dataclasses import dataclass
from nicegui import ui

@dataclass
class AppState:
    picker_open: bool = False # 与菜单状态绑定的属性

state = AppState()

# 颜色选择器菜单状态与 state.picker_open 双向绑定
picker = ui.color_picker().bind_value(state, 'picker_open')

# 显示当前菜单状态
ui.label().bind_text_from(state, 'picker_open', lambda open: f'菜单状态: {"打开" if open else "关闭"}')

# 按钮修改绑定属性（间接控制菜单）
ui.button('切换菜单状态', on_click=lambda: setattr(state, 'picker_open', not state.picker_open))
```

4. 样式与扩展方法

用于自定义组件外观、添加资源或辅助功能：

方法名	说明
<code>tooltip(text: str)</code>	为组件添加 tooltip 提示（鼠标悬浮时显示文本，如 "选择颜色"）
<code>add_resource(path)</code>	为组件添加资源文件（如自定义 CSS/JS，用于扩展颜色选择器功能）
<code>default_classes(...)</code>	全局修改该类组件的默认 HTML 类（需在实例化前调用，如统一设置菜单样式）

方法名	说明
<code>default_style(...)</code>	全局修改该类组件的默认 CSS 样式（需在实例化前调用）
<code>mark(*markers)</code>	为组件添加标记（用于测试查询或依赖管理）

样式自定义示例

```
# 全局修改颜色选择器菜单的默认样式（实例化前调用）
ui.color_picker.default_style(add='width: 300px; border-radius: 8px;')
ui.color_picker.default_classes(add='shadow-lg border-2 border-gray-200')

# 实例化时添加局部样式
with ui.button('自定义样式选择器'):
    ui.color_picker(
        style='padding: 10px;', # 内联样式
        on_pick=lambda e: ui.notify(f'选中颜色: {e.color}')
    ).tooltip('点击选择颜色') # 添加 tooltip
```

5. 其他实用方法

方法名	说明
<code>clear()</code>	清除组件的所有子元素（极少用于颜色选择器，适用于嵌套复杂内容场景）
<code>delete()</code>	删除组件及其所有子元素（释放资源，生命周期结束）
<code>ancestors(include_self)</code>	迭代组件的祖先元素（ <code>include_self=True</code> 包含自身）
<code>descendants(include_self)</code>	迭代组件的子元素（ <code>include_self=True</code> 包含自身）
<code>update()</code>	强制在客户端更新组件状态（用于手动同步数据，如动态修改样式后刷新）

四、关键特性：颜色选择器自定义（q_color 属性）

由于 `ui.color_picker` 的核心颜色选择功能基于 Quasar 的 `QColor` 组件，且 `QColor` 嵌套在 `QMenu` 内部，**无法直接通过 `props` 方法修改 `QColor` 的属性**，需通过组件的 `q_color` 属性间接配置 `QColor` 的原生参数，实现颜色选择器的功能自定义。

常用 QColor 配置参数

QColor 参数	说明
<code>default-view</code>	默认视图类型（ <code>palette</code> 调色板视图、 <code>gradient</code> 渐变视图、 <code>wheel</code> 色轮视图）
<code>no-header</code>	隐藏 QColor 组件的头部（去除标题、关闭按钮）
<code>no-footer</code>	隐藏 QColor 组件的底部（去除确认、取消按钮）
<code>format</code>	颜色输出格式（ <code>hex</code> 十六进制、 <code>rgb</code> 、 <code>hsl</code> 等）

QColor 参数	说明
<code>predefined-colors</code>	自定义预设颜色列表（如 <code>["#ff0000", "#00ff00", "#0000ff"]</code> ）

自定义 QColor 示例

```
from nicegui import ui

with ui.button(icon='palette', label='高级颜色选择'):
    picker = ui.color_picker(
        on_pick=lambda e: ui.notify(f'选中颜色（调色板模式）：{e.color}'),
        value=False # 初始关闭菜单
    )
    # 配置 QColor 组件：默认调色板视图、隐藏头部和底部、指定颜色格式为 hex
    picker.q_color.props('default-view=palette no-header no-footer format=hex')
    # 设置预设颜色
    picker.q_color.props('predefined-colors=["#ff0000", "#00ff00", "#0000ff", "#ffff00",
"#800080"]')

ui.run()
```

五、事件处理

1. 核心事件：颜色拾取（on_pick）

颜色拾取事件是 `ui.color_picker` 的核心事件，支持两种绑定方式（等价）：

```
# 方式1：初始化时通过 on_pick 参数绑定
with ui.button('选择颜色'):
    ui.color_picker(on_pick=lambda e: print(f'拾取颜色：{e.color}'))

# 方式2：通过 on_pick 方法绑定
with ui.button('选择颜色'):
    picker = ui.color_picker()
    picker.on_pick(lambda e: print(f'拾取颜色：{e.color}'))
```

2. 菜单状态变更事件（on_value_change）

监听菜单打开 / 关闭状态的变更：

```
with ui.button('选择颜色'):
    ui.color_picker().on_value_change(
        lambda e: print(f'菜单状态变更为：{"打开" if e.value else "关闭"}')
    )
```

3. 通用事件订阅（on 方法）

通过 `on()` 方法订阅其他 DOM 事件（如点击、鼠标悬浮等），支持客户端 JS 处理或服务端 Python 处理：

```
with ui.button('选择颜色'):  
    ui.color_picker()  
    # 订阅菜单打开事件（客户端 JS 处理）  
    .on('show', js_handler='() => console.log("菜单打开了")')  
    # 订阅菜单关闭事件（服务端 Python 处理）  
    .on('hide', lambda: print('菜单关闭了'))
```

六、版本兼容性说明

特性 / 属性	支持版本
<code>html_id</code>	2.16.0+
<code>toggle</code> 参数 (<code>default_classes</code>)	2.7.0+
<code>strict</code> 参数 (绑定方法)	3.0.0+
同时指定 Python 和 JS 事件处理 (<code>on</code> 方法)	2.18.0+

七、与 ui.color_input 的核心区别

特性	ui.color_picker	ui.color_input
交互形式	弹窗菜单（需触发元素）	输入框 + 下拉选择器（inline 形式）
核心场景	节省界面空间、快捷颜色选择	表单嵌入、需显示颜色值的场景
自定义能力	支持 QColor 组件深度配置	侧重输入框样式和预览配置
初始参数	仅 <code>on_pick</code> 、 <code>value</code> 两个核心参数	多参数（ <code>label</code> 、 <code>placeholder</code> 等）

简单来说：需要嵌入表单、显示颜色值时用 `ui.color_input`；需要节省空间、提供独立颜色选择面板时用 `ui.color_picker`。

总结

`ui.color_picker` 是一款专注于弹窗式颜色选择的组件，核心优势在于：

- 1. 基于 Quasar 组件，交互流畅且跨浏览器兼容；
- 2. 菜单状态控制灵活，支持手动打开 / 关闭、双向绑定；
- 3. 支持 QColor 组件深度自定义，满足复杂颜色选择需求；
- 4. 样式扩展能力强，可通过类、内联样式、全局配置适配不同界面风格。

适用于工具栏、快捷操作面板、精简版设置界面等场景，结合 NiceGUI 的按钮、图标等组件可快速实现直观的颜色选择交互。